WEEK 2 · DAY 11 · DIARY TASK

Signal Preprocessing

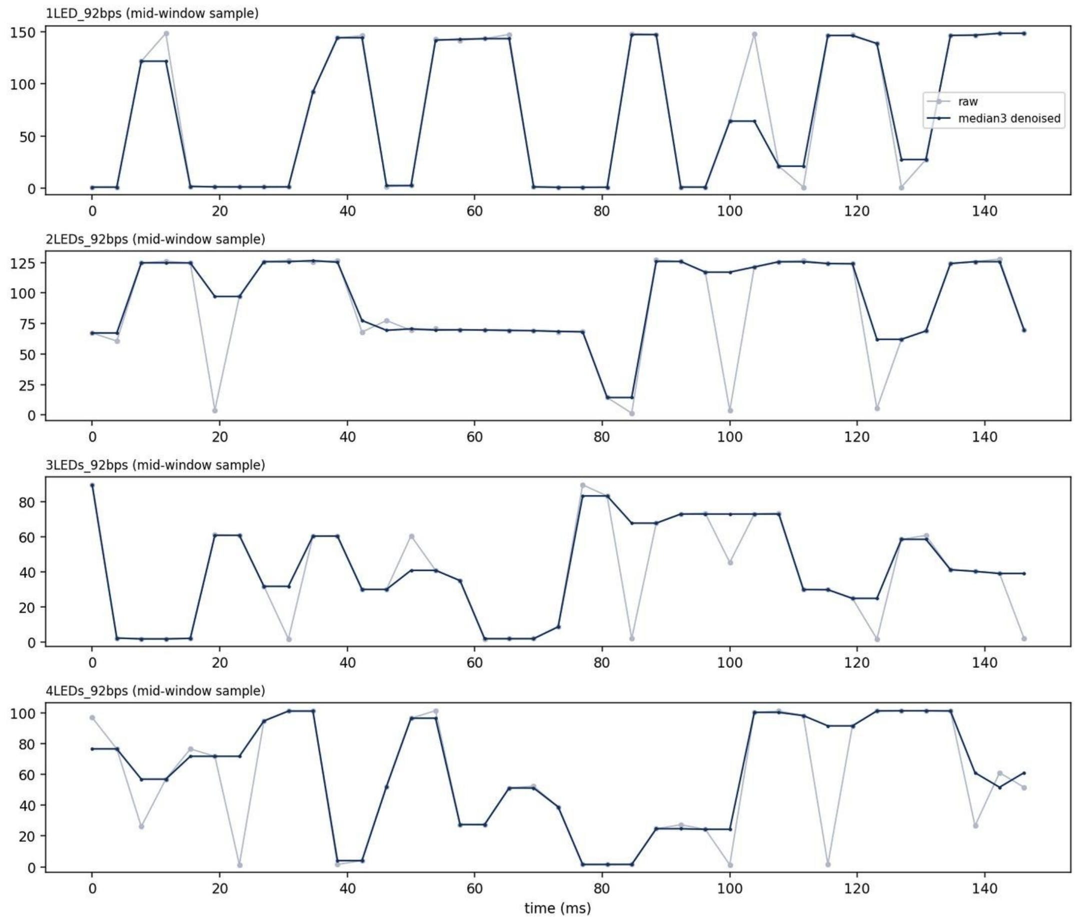
1. PIPELINE OVERVIEW

Raw video → fixed ROI (Day 9) → per-frame ROI mean → 3-sample median filter → per-video min-max normalization. Implemented in preprocess.py, run end-to-end on all 4 videos' full active windows.

File	Active Window	Frames Processed	Raw Range
1LED_92bps.mp4	0.5s → 107.5s	27,820	0.8 – 150.3
2LEDs_92bps.mp4	3.5s → 69.5s	17,160	0.9 – 129.3
3LEDs_92bps.mp4	3.5s → 47.5s	11,440	1.3 – 91.7
4LEDs_92bps.mp4	7.0s → 39.5s	8,450	1.0 – 103.5

2. WHY DENOISING IS DELIBERATELY MINIMAL

Day 10 found the per-frame signal is already under sampled relative to the bit rate (2.826 frames/bit) and that SNR is very high (57x–318x background noise). A large smoothing window would blur or erase real bit transitions, not just noise, so only a 3-sample median filter is applied, which removes single-frame dropout spikes while leaving multi-frame plateau structure intact.



Visible effect: the filter corrects isolated single-frame dropouts (e.g. 2-LED video at ~20ms, ~85ms, ~100ms, where one frame spikes to near-zero mid-plateau) without altering the surrounding plateau shape.

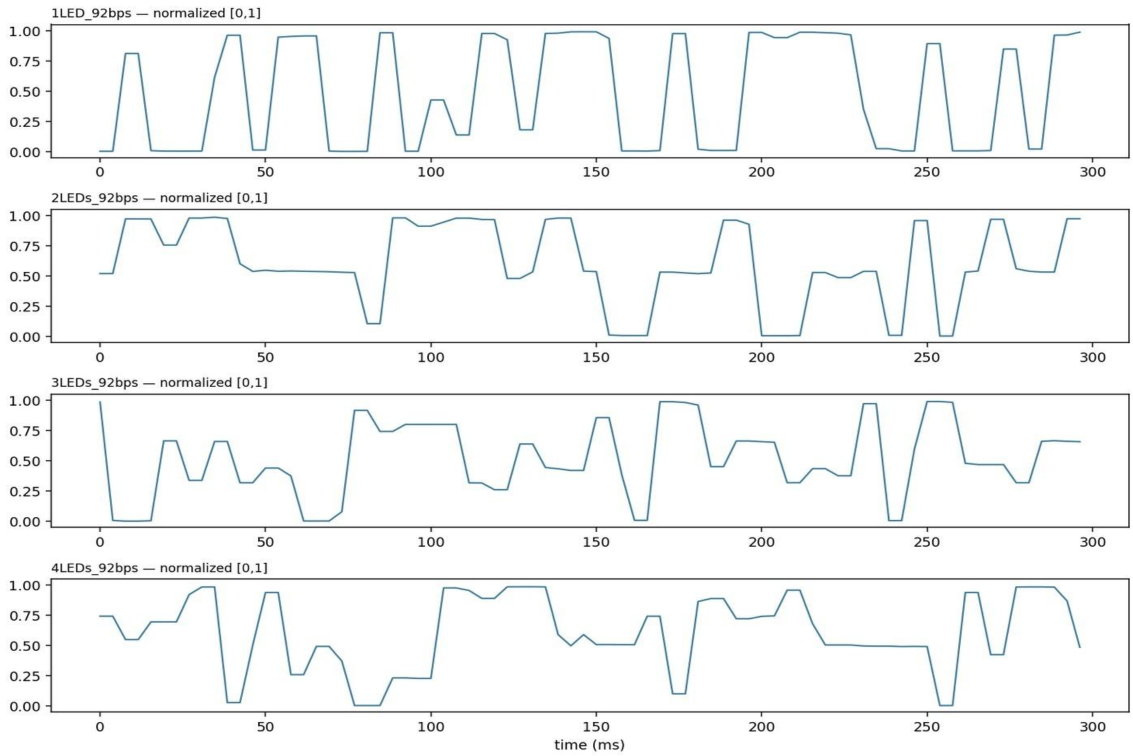
3. HOW MUCH THE FILTER ACTUALLY CHANGES

File	Frames Altered	Median Change	Mean Change (% of range)
1LED_92bps.mp4	51.0%	0.55	6.2%
2LEDs_92bps.mp4	49.2%	1.15	10.2%
3LEDs_92bps.mp4	48.7%	0.75	9.4%
4LEDs_92bps.mp4	46.0%	1.50	12.3%

Being transparent about this rather than just calling it "light": about half of all frames see some change, but the median change is small (under 1.5 units on a 0-150 scale) — most "changes" are tiny. The filter's real value is in the long tail: a small number of frames get corrected by a large amount, and those are exactly the single-frame dropout spikes visible in Section 2, not genuine signal being smoothed away.

4. NORMALIZATION

Per-video min-max normalization to [0,1], computed over the active window only (dead-time excluded). Necessary because raw ROI intensity ranges differ purely due to LED count/distance (1-LED peaks ~150, 3-LED peaks ~92) — without this, downstream thresholding logic would need a different absolute cutoff per video, which is fragile and not generalizable.



5. AN ADDITIONAL FINDING: RAMP-UP AT ACTIVE-WINDOW START

The Day 7 active-window boundary was defined by a 0.5s-window threshold crossing — coarse enough that the exact start of the window is not necessarily full-amplitude transmission. For example, the 1-

LED video's first ~150ms inside the nominal active window averages 38.3 (vs. 75.3 mid-window) — a real ramp-up transient, not a processing artifact. This doesn't affect the saved processed signal (which covers the full window correctly) but is worth flagging: any Day 13 synchronization work that assumes uniform signal quality from the very first sample of the active window should account for this.

6. DETERMINISM & REPRODUCIBILITY

- ROI boxes and active windows are fixed constants (frozen Day 9/Day 7), not re-detected per run.
- Frame reads are sequential from a fixed start index — no seeking mid-extraction, which was the same method Day 7 used to verify readability.
- All settings (fps, denoise method, normalize method, ROI, active window) are recorded in preprocessing_config. json — re-running preprocess.py against the same raw files reproduces identical output.
- Raw, denoised, and normalized signals are all saved separately (not just the final normalized output) so any downstream stage can fall back to an earlier processing step if needed.

7. DELIVERABLES

- preprocess.py — the deterministic pipeline (this document describes and justifies it)
- processed_signals.csv — long-format export: file, frame_idx, time_ms, roi_raw, roi_denoised, roi_normalized (64,870 rows)
- preprocessing_config.json — exact settings and per-video stats for reproducibility

8. NEXT STEP

Day 12 (Signal Pipeline Review, Mentor Review 4) presents this pipeline and its design decisions for sign-off — the ramp-up finding in Section 5 and the denoise transparency in Section 3 are worth surfacing there rather than glossing over.